CAPÍTULO

16 ASEGURAMIENTO DE LA CALIDAD DEL SOFTWARE

CONCEPTOS CLAVE

confiabilidad del software . . 376
elementos del ACS. 370
  estadístico 374
  plan 379
  tareas 371
enfoques formales. 373
estándar ISO 9001-2000 . . 378
metas. 371
seguridad del software 378
Seis Sigma 375

El enfoque de la ingeniería de software descrito en este libro se dirige a una sola meta: producir software a tiempo y de alta calidad. Pero muchos lectores se preguntarán: "¿Qué es calidad del software?".

Philip Crosby [Cro79], en su libro clásico sobre calidad, da una respuesta irónica a esta pregunta:

> El problema de la administración de la calidad no es lo que la gente ignora de ella. El problema es lo que piensan que saben...
>
> En ese sentido, la calidad tiene mucho en común con el sexo. Todo mundo lo busca (en ciertas condiciones, por supuesto). Todos creen que lo entienden (aunque no querrían explicarlo). Todos piensan que su ejecución sólo consiste en seguir las inclinaciones naturales (después de todo, lo hacemos de algún modo). Y, por supuesto, la mayoría de la gente siente que los problemas en esta área los causan las demás personas (si sólo se dieran el tiempo de hacer las cosas bien).

En realidad, la calidad es un concepto difícil (se abordó con cierto detalle en el capítulo 14).[1]

Algunos desarrolladores de software todavía creen que la calidad del software es algo por lo que hay que empezar a preocuparse una vez generado el código. Nada podría estar más lejos de la verdad... El *aseguramiento de la calidad del software* (con frecuencia llamado *administración*

UNA MIRADA RÁPIDA

¿Qué es? No basta hablar por hablar para decir que la calidad del software es importante. Tiene que 1) definirse explícitamente lo que quiere decir "calidad del software", 2) crearse un conjunto de actividades que ayuden a garantizar que todo producto de la ingeniería de software tenga alta calidad, 3) desarrollarse el control de calidad y las actividades para asegurar ésta en todo proyecto de software, 4) usarse métricas para desarrollar estrategias a fin de mejorar el proceso del software y, en consecuencia, la calidad del producto final.

¿Quién lo hace? Todos los involucrados en el proceso de ingeniería de software son los responsables de la calidad.

¿Por qué es importante? Las cosas pueden hacerse bien o pueden volverse a hacer. Si un equipo de software pone el énfasis en la calidad en todas las actividades de la ingeniería de software, se reduce la cantidad de repeticiones que debe hacer. Eso da como resultado costos más bajos y, lo que es más importante, un mejor tiempo para llegar al mercado.

¿Cuáles son las etapas? Antes de iniciar las actividades de aseguramiento de la calidad del software (ACS), es importante definir la *calidad del software* en varios niveles diferentes de abstracción. Una vez que se entiende lo que es la calidad, el equipo de software debe identificar un conjunto de actividades de ACS que filtren los errores de los productos del trabajo antes de que se aprueben.

¿Cuál es el producto final? Se crea un Plan de Aseguramiento de la Calidad del Software para definir una estrategia de ACS del equipo. Durante la modelación y codificación, el producto principal del ACS es la salida de las revisiones técnicas (véase el capítulo 15). Durante las pruebas (capítulos 17 a 20), se generan los planes y procedimientos de prueba, así como otros productos del trabajo asociados con el proceso de mejora.

¿Cómo me aseguro de que lo hice bien? Hay que encontrar los errores antes de que se vuelvan defectos... Es decir, debe trabajarse para mejorar la eficiencia en la eliminación de defectos (capítulo 23), a fin de reducir la cantidad de repeticiones que tenga que hacer el equipo del software.

1 Si no ha leído el capítulo 14, debe leerlo ahora.

de la calidad) es una actividad sombrilla (véase el capítulo 2) que se aplica en todo el proceso del software.

El aseguramiento de la calidad del software (ACS) incluye lo siguiente: 1) un proceso de ACS, 2) tareas específicas de aseguramiento y control de la calidad (incluidas revisiones técnicas y una estrategia de pruebas relacionadas entre sí), 3) prácticas eficaces de ingeniería de software (métodos y herramientas), 4) control de todos los productos del trabajo de software y de los cambios que sufren (véase el capítulo 22), 5) un procedimiento para garantizar el cumplimiento de los estándares del desarrollo de software (cuando sea aplicable) y 6) mecanismos de medición y reporte.

Este capítulo se centra en aspectos de la administración y en las actividades específicas del proceso que permiten a una organización de software garantizar que hace "las cosas correctas en el momento correcto y de la forma correcta".

16.1 Antecedentes

El control y aseguramiento de la calidad son actividades esenciales para cualquier negocio que genere productos que utilicen otras personas. Antes del siglo XX, el control de calidad era responsabilidad única del artesano que elaboraba el producto. Cuando pasó el tiempo y las técnicas de la producción en masa se hicieron comunes, el control de calidad se convirtió en una actividad ejecutada por personas diferentes de aquellas que elaboraban el producto.

Cita:

"Cometes demasiados errores equivocados."

Yogi Berra

La primera función formal de aseguramiento y control de la calidad se introdujo en los laboratorios Bell en 1916 y se difundió con rapidez al resto del mundo de la manufactura. Durante la década de 1940, sugirieron enfoques más formales del control de calidad. Éstos se basaban en la medición y en el proceso de la mejora continua [Dem86] como elementos clave de la administración de la calidad.

Actualmente, toda compañía tiene mecanismos para asegurar la calidad en sus productos. En realidad, en las últimas décadas, las afirmaciones explícitas del compromiso de una compañía con la calidad se han vuelto un mantra de la mercadotecnia.

La historia del aseguramiento de la calidad en el desarrollo del software corre de manera paralela con la historia de la calidad en la manufactura del hardware. En los primeros días de la computación (décadas de 1950 y 1960), la calidad era responsabilidad única del programador. Los estándares para asegurar la calidad del software se introdujeron en los contratos para desarrollar software militar en la década de 1970 y se extendieron con rapidez al desarrollo de software en el mundo comercial [IEE93]. Si se amplía la definición presentada al principio, el aseguramiento de la calidad del software es un "patrón planeado y sistemático de acciones" [Sch98c] que se requieren para garantizar alta calidad en el software. El alcance de la responsabilidad del aseguramiento de la calidad se caracteriza mejor si se parafrasea un comercial de un automóvil popular: "La calidad es el empleo número 1." La implicación para el software es que muchas entidades diferentes tienen responsabilidad en el aseguramiento de la calidad del software: ingenieros de software, gerentes de proyecto, clientes, vendedores y los individuos que trabajan en el grupo de ACS.

El grupo de ACS funciona como representante del cliente en el interior de la empresa. Es decir, la gente que realiza el ACS debe ver al software desde el punto de vista del cliente. ¿El software cumple adecuadamente los factores de calidad mencionados en el capítulo 14? ¿El desarrollo del software se condujo de acuerdo con estándares preestablecidos? ¿Las disciplinas técnicas han cumplido con sus roles como parte de la actividad de ACS? El grupo de ACS trata de responder éstas y otras preguntas para garantizar que se mantenga la calidad del software.

16.2 Elementos de aseguramiento de la calidad del software

WebRef

En la dirección **www.swqual.com/newsletter/vol2/no1/vol2no1.html**, se encuentra un análisis profundo del ACS, que incluye una amplia variedad de definiciones.

El aseguramiento de la calidad del software incluye un rango amplio de preocupaciones y actividades que se centran en la administración de la calidad del software. Éstas se resumen como sigue [Hor03]:

Estándares. El IEEE, ISO y otras organizaciones que establecen estándares han producido una amplia variedad de ellos para ingeniería de software y documentos relacionados. Los estándares los adopta de manera voluntaria una organización de software o los impone el cliente u otros participantes. El trabajo del ACS es asegurar que los estándares que se hayan adoptado se sigan, y que todos los productos del trabajo se apeguen a ellos.

Revisiones y auditorías. Las revisiones técnicas son una actividad del control de calidad que realizan ingenieros de software para otros ingenieros de software (véase el capítulo 15). Su objetivo es detectar errores. Las auditorías son un tipo de revisión efectuada por personal de ACS con objeto de garantizar que se sigan los lineamientos de calidad en el trabajo de la ingeniería de software. Por ejemplo, una auditoría del proceso de revisión se efectúa para asegurar que las revisiones se lleven a cabo de manera que tengan la máxima probabilidad de descubrir errores.

Pruebas. Las pruebas del software (capítulos 17 a 20) son una función del control de calidad que tiene un objetivo principal: detectar errores. El trabajo del ACS es garantizar que las pruebas se planeen en forma apropiada y que se realicen con eficiencia, de modo que la probabilidad de que logren su objetivo principal sea máxima.

Colección y análisis de los errores. La única manera de mejorar es medir cómo se está haciendo algo. El ACS reúne y analiza errores y datos acerca de los defectos para entender mejor cómo se cometen los errores y qué actividades de la ingeniería de software son más apropiadas para eliminarlos.

Administración del cambio. El cambio es uno de los aspectos que más irrumpe en cualquier proyecto de software. Si no se administra en forma adecuada, lleva a la confusión y ésta casi siempre genera mala calidad. El ACS asegura que se hayan instituido prácticas adecuadas de administración del cambio (véase el capítulo 22).

Educación. Toda organización de software quiere mejorar sus prácticas de ingeniería de software. Un contribuyente clave de la mejora es la educación de los ingenieros de software, de sus gerentes y de otros participantes. La organización de ACS lleva el liderazgo en la mejora del proceso de software (capítulo 30) y es clave para proponer y patrocinar programas educativos.

Cita:

"La excelencia es la capacidad ilimitada de mejorar la calidad de lo que se tenga para ofrecer."

Rick Petin

Administración de los proveedores. Son tres las categorías de software que se adquieren a proveedores externos: *paquetes contenidos en una caja* (por ejemplo, *Office*, de Microsoft); *un shell personalizado* [Hor03], que da una estructura básica, tipo esqueleto, que se adapta de manera única a las necesidades del comprador; y *software contratado*, que se diseña y construye especialmente a partir de especificaciones provistas por la organización cliente. El trabajo de la organización de ACS es garantizar que se obtenga software de alta calidad a partir de las sugerencias de prácticas específicas de calidad que el proveedor debe seguir (cuando sea posible) y de la incorporación de cláusulas de calidad como parte de cualquier contrato con un proveedor externo.

Administración de la seguridad. Con el aumento de los delitos cibernéticos y de las nuevas regulaciones gubernamentales respecto de la privacidad, toda organización de software debe instituir políticas para proteger los datos en todos los niveles, establecer cortafuegos de protección para las *webapps* y asegurar que el software no va a ser vulnerado in-

ternamente. El ACS garantiza que para lograr la seguridad del software, se utilicen el proceso y la tecnología apropiados.

Seguridad. Debido a que el software casi siempre es un componente crucial de los sistemas humanos (como aplicaciones automotrices o aeronáuticas), la consecuencia de defectos ocultos puede ser catastrófica. El ACS es responsable de evaluar el efecto de las fallas del software y de dar los pasos que se requieren para disminuir el riesgo.

Administración de riesgos. Aunque el análisis y la mitigación de riesgos (véase el capítulo 28) es asunto de los ingenieros de software, la organización del ACS garantiza que las actividades de administración de riesgos se efectúen en forma apropiada y que se establezcan planes de contingencia relacionados con los riesgos.

Además de cada una de estas preocupaciones y actividades, el ACS tiene como preocupación dominante asegurar que las actividades de apoyo del software (como mantenimiento, líneas de ayuda, documentación y manuales) se lleven a cabo o se produzcan con calidad.

INFORMACIÓN



Recursos para la administración de la calidad

En la red mundial existen decenas de recursos para la administración de la calidad, incluidas sociedades profesionales, organizaciones emisoras de estándares y fuentes de información general. Los sitios siguientes constituyen un buen punto de partida:

American Society for Quality (ASQ), División de Software, **www.asq.org/software**
Association for Computer Machinery, **www.acm.org**, Centro de Datos y Análisis del Software (DACS), **www.dacs.dtic.mil/**
International Organization for Standardization (ISO), **www.iso.ch**
ISO SPIECE, **www.isospiece.com**
Malcolm Baldridge National Quality Award, **www.quality.nist.gov**
Software Engineering Institute, **www.sei.cmu.edu/**
Software Testing and Quality Engineering, **www.stickyminds.com**
Six Sigma Resources, **www.isixsigma.com/** **www.asq.org/sixsigma/**
TickIT International, temas sobre certificación de la calidad, **www.tickit.org/international.htm**
Total Quality Management (TQM)
- Información general: **www.gslis.utexas.edu/~rpollock/tqm.html**
- Artículos: **www.work911.com/tqmarticles.htm**
- Glosario: **www.quality.org/TQM-MSI/TQM-glossary.html**

16.3 TAREAS, METAS Y MÉTRICAS DEL ACS

El aseguramiento de la calidad del software se compone de varias tareas asociadas con dos entidades diferentes: los ingenieros de software que hacen el trabajo técnico y un grupo de ACS que tiene la responsabilidad de planear, supervisar, registrar, analizar y hacer reportes acerca de la calidad.

Los ingenieros de software abordan la calidad (y ejecutan actividades para controlarla), aplicando métodos y medidas técnicas sólidos, realizando revisiones técnicas y haciendo pruebas de software bien planeadas.

16.3.1 Tareas del ACS

El objetivo del grupo de ACS es auxiliar al equipo del software para lograr un producto final de alta calidad. El Instituto de Ingeniería de Software recomienda un conjunto de acciones de ACS que se dirigen a la planeación, supervisión, registro, análisis y elaboración de reportes para el aseguramiento de la calidad. Estas acciones son realizadas (o facilitadas) por un grupo independiente de ACS que hace lo siguiente:

¿Cuál es el rol de un grupo de ACS?

Prepara el plan de ACS para un proyecto. El plan se desarrolla como parte de la preparación del proyecto y es revisado por todos los participantes. Las acciones de aseguramiento de la calidad efectuadas por el equipo de ingeniería de software y por el grupo de ACS son dirigidas por el plan. Éste identifica las evaluaciones que se van a realizar, las auditorías y revisiones por efectuar, los estándares aplicables al proyecto, los procedimientos para reportar y dar seguimiento a los errores, los productos del trabajo que genera el grupo de ACS y la retroalimentación que se dará al equipo del software.

Participa en el desarrollo de la descripción del software del proyecto. El equipo de software selecciona un proceso para el trabajo que se va a realizar. El grupo de ACS revisa la descripción del proceso a fin de cumplir con la política organizacional, los estándares internos para el software, los estándares impuestos desde el exterior (como la norma ISO-9001) y otras partes del plan del proyecto de software.

Revisa las actividades de la ingeniería de software a fin de verificar el cumplimiento mediante el proceso definido para el software. El grupo de ACS identifica, documenta y da seguimiento a las desviaciones del proceso y verifica que se hayan hecho las correcciones pertinentes.

Audita los productos del trabajo de software designados para verificar que se cumpla con aquellos definidos como parte del proceso de software. El grupo de ACS revisa productos del trabajo seleccionados; identifica, documenta y da seguimiento a las desviaciones; verifica que se hayan hecho las correcciones necesarias y reporta periódicamente los resultados de su trabajo al gerente del proyecto.

Asegura que las desviaciones en el trabajo de software y sus productos se documenten y manejen de acuerdo con un procedimiento documentado. Las desviaciones pueden encontrarse en el plan del proyecto, la descripción del proceso, los estándares aplicables o los productos del trabajo de la ingeniería de software.

Registra toda falta de cumplimiento y la reporta a la alta dirección. Se da seguimiento a los incumplimientos hasta que son resueltos.

Además de estas acciones, el grupo de ACS coordina el control y administración del cambio (véase el capítulo 22) y ayuda a recabar y analizar métricas para el software.

16.3.2 Metas, atributos y métricas

Cita:

"La calidad nunca es un accidente; siempre es el resultado de una intención clara, un esfuerzo sincero, una dirección inteligente y una ejecución hábil; representa la elección sabia de muchas alternativas".

William A. Foster

Las acciones de ACS descritas en la sección anterior se realizan con objeto de alcanzar un conjunto de metas pragmáticas:

Calidad de los requerimientos. La corrección, completitud y consistencia del modelo de requerimientos tendrá una gran influencia en la calidad de todos los productos del trabajo que sigan. El ACS debe garantizar que el equipo de software ha revisado en forma apropiada el modelo de requerimientos a fin de alcanzar un alto nivel de calidad.

Calidad del diseño. Todo elemento del modelo del diseño debe ser evaluado por el equipo del software para asegurar que tenga alta calidad y que el diseño en sí se apegue a los requerimientos. El ACS busca atributos del diseño que sean indicadores de la calidad.

Calidad del código. El código fuente y los productos del trabajo relacionados (por ejemplo, otra información descriptiva) deben apegarse a los estándares locales de codificación y tener características que faciliten darle mantenimiento. El ACS debe identificar aquellos atributos que permitan hacer un análisis razonable de la calidad del código.

Eficacia del control de calidad. Un equipo de software debe aplicar recursos limitados, en forma tal que tenga la máxima probabilidad de lograr un resultado de alta calidad. El

FIGURA 16.1 **Metas atributos y métricas de la calidad del software**
Fuente: Adaptado de [Hya96].

Meta	Atributo	Métrica
Calidad de los requerimientos	Ambigüedad	Número de modificadores ambiguos (por ejemplo, muchos, grande, amigable, etc.)
	Completitud	Número de TBA y TBD
	Comprensibilidad	Número de secciones y subsecciones
	Volatilidad	Número de cambios por requerimiento
		Tiempo (por actividad) cuando se solicita un cambio
	Trazabilidad	Número de requerimientos no trazables hasta el diseño o código
	Claridad del modelo	Número de modelos UML
		Número de páginas descriptivas por modelo
		Número de errores de UML
Calidad del diseño	Integridad arquitectónica	Existencia del modelo arquitectónico
	Completitud de componentes	Número de componentes que se siguen hasta el modelo arquitectónico
		Complejidad del diseño del procedimiento
	Complejidad de la interfaz	Número promedio de pasos para llegar a una función o contenido normal
		Distribución apropiada
	Patrones	Número de patrones utilizados
Calidad del código	Complejidad	Complejidad ciclomática
	Facilidad de mantenimiento	Factores de diseño (capítulo 8)
	Comprensibilidad	Porcentaje de comentarios internos
		Convenciones variables de nomenclatura
	Reusabilidad	Porcentaje de componentes reutilizados
	Documentación	Índice de legibilidad
Eficacia del control de calidad	Asignación de recursos	Porcentaje de personal por hora y por actividad
	Tasa de finalización	Tiempo de terminación real *versus* lo planeado
	Eficacia de la revisión	Ver medición de la revisión (capítulo 14)
	Eficacia de las pruebas	Número de errores de importancia crítica encontrados
		Esfuerzo requerido para corregir un error
		Origen del error

ACS analiza la asignación de recursos para las revisiones y pruebas a fin de evaluar si se asignan en la forma más eficaz.

La figura 16.1 (adaptada de [Hya96]) identifica los atributos que son indicadores de la existencia de la calidad para cada una de las metas mencionadas. También se presentan las métricas que se utilizan para indicar la fortaleza relativa de un atributo.

16.4 ENFOQUES FORMALES AL ACS

En las secciones anteriores, se dijo que la calidad del software es el trabajo de cada quien y que puede lograrse por medio de una práctica competente de la ingeniería de software, así como de la aplicación de revisiones técnicas, de una estrategia de pruebas con relaciones múltiples, de un mejor control de los productos del trabajo de software y de los cambios efectuados sobre

WebRef

En la dirección **www.gslis.utexas.edu/~rpollock/tqm.html**, se encuentra información útil acerca del ACS y de los métodos formales de la calidad.

ellos, así como de la aplicación de estándares aceptados de la ingeniería de software. Además, la calidad se define en términos de una amplia variedad de atributos de la calidad y se mide (indirectamente) con el empleo de varios índices y métricas.

En las últimas tres décadas, un segmento pequeño pero sonoro de la comunidad de la ingeniería de software ha afirmado que se requiere un enfoque más formal para el ACS. Puede decirse que un programa de cómputo es un objeto matemático. Para cada lenguaje de programación, es posible definir una sintaxis y semántica rigurosas, y se dispone de un enfoque igualmente riguroso para la especificación de los requerimientos del software (véase el capítulo 21). Si el modelo de los requerimientos (especificación) y el lenguaje de programación se representan en forma rigurosa, debe ser posible usar una demostración matemática para la corrección, de modo que se confirme que un programa se ajusta exactamente a sus especificaciones.

Los intentos de demostrar la corrección de un programa no son nuevos. Dijkstra [Dij76a] y Linger, Mills y Witt [Lin79], entre otros, han invocado pruebas de la corrección de programas y las han relacionado con el uso de conceptos de programación estructurada (véase el capítulo 10).

16.5 Aseguramiento estadístico de la calidad del software

El aseguramiento estadístico de la calidad del software refleja una tendencia creciente en la industria para que se vuelva más cuantitativo respecto de la calidad. Para el software, el aseguramiento estadístico de la calidad implica los pasos siguientes:

? ¿Qué pasos se requieren para efectuar el ACS estadístico?

1. Se recaba y clasifica la información acerca de errores y defectos del software.
2. Se hace un intento por rastrear cada error y defecto hasta sus primeras causas (por ejemplo, no conformidad con las especificaciones, error de diseño, violación de los estándares, mala comunicación con el cliente, etc.).
3. Con el uso del Principio de Pareto (80 por ciento de los defectos se debe a 20 por ciento de todas las causas posibles), se identifica 20 por ciento de las causas de errores y defectos (las *pocas vitales*).
4. Una vez identificadas las pocas causas vitales, se corrigen los problemas que han dado origen a los errores y defectos.

Este concepto relativamente simple representa un paso importante hacia la creación de un proceso adaptativo del software en el que se hacen cambios para mejorar aquellos elementos del proceso que introducen errores.

16.5.1 Ejemplo general

Cita:

"Un análisis estadístico, si se realiza en forma apropiada, es una disección delicada de las incertidumbres, una cirugía de las suposiciones."

M. J. Moroney

A fin de ilustrar el uso de los métodos estadísticos para el trabajo de ingeniería de software, suponga que una organización de ingeniería de software recaba información sobre los errores y defectos cometidos en un periodo de un año. Algunos de dichos errores se descubren a medida que se desarrolla el software. Otros (defectos) se encuentran después de haber liberado el software a sus usuarios finales. Aunque se descubren cientos de problemas diferentes, todos pueden rastrearse hasta una (o más) de las causas siguientes:

- Especificaciones erróneas o incompletas (EEI)
- Mala interpretación de la comunicación con el cliente (MCC)
- Desviación intencional de las especificaciones (DIE)
- Violación de los estándares de programación (VEP)
- Error en la representación de los datos (ERD)

FIGURA 16.2
Colección de datos para hacer ACS estadístico

	Total		Serio		Moderado		Menor	
Error	**No.**	**%**	**No.**	**%**	**No.**	**%**	**No.**	**%**
IES	205	22%	34	27%	68	18%	103	24%
MCC	156	17%	12	9%	68	18%	76	17%
IDS	48	5%	1	1%	24	6%	23	5%
VPS	25	3%	0	0%	15	4%	10	2%
EDR	130	14%	26	20%	68	18%	36	8%
ICI	58	6%	9	7%	18	5%	31	7%
EDL	45	5%	14	11%	12	3%	19	4%
IET	95	10%	12	9%	35	9%	48	11%
IID	36	4%	2	2%	20	5%	14	3%
PLT	60	6%	15	12%	19	5%	26	6%
HCI	28	3%	3	2%	17	4%	8	2%
MIS	56	6%	0	0%	15	4%	41	9%
Totales	942	100%	128	100%	379	100%	435	100%

- Interfaz componente inconsistente (ICI)
- Error en el diseño lógico (EDL)
- Pruebas incompletas o erróneas (PIE)
- Documentación inexacta o incompleta (DII)
- Error en la traducción del lenguaje de programación del diseño (LPD)
- Interfaz humano/computadora ambigua o inconsistente (IHC)
- Varios (V)

Cita:

"20 por ciento del código tiene 80 por cierto de los errores. Encuéntrelos, corríjalos".

Lowell Arthur

Para aplicar el ACS estadístico, se elabora la tabla de la figura 16.2. La tabla indica que EEI, MCC y ERD son las pocas causas vitales que originan 53 por ciento de todos los errores. Sin embargo, debe notarse que EEI, ERD, LPD y EDL se habrían seleccionado como las pocas causas vitales si se consideran sólo errores serios. Una vez que las pocas causas vitales han sido determinadas, la organización de ingeniería de software comienza su acción correctiva. Por ejemplo, a fin de corregir el MCC, deben implementarse técnicas para recabar requerimientos (capítulo 5) que mejoren la calidad de la comunicación y las especificaciones con el cliente. Para mejorar el ERD, deben adquirirse herramientas para desarrollar la modelación de casos y realizar datos y revisiones del diseño más significativos.

Es importante notar que la acción correctiva se centra sobre todo en las pocas causas vitales. En tanto éstas se corrigen, nuevas candidatas se van a la cumbre de la pila.

Las técnicas para el aseguramiento correctivo han sido propuestas para dar una mejora sustancial de la calidad [Art97]. En ciertos casos, las organizaciones de software han tenido una reducción anual de 50 por ciento en defectos después de aplicar esta técnica.

La aplicación del ACS estadístico y el Principio de Pareto se resumen en una sola oración: *Pasa tu tiempo viendo las cosas que realmente importan, pero primero asegúrate de que entiendes lo que realmente importa...*

16.5.2 Seis Sigma para la ingeniería de software

Seis Sigma es la estrategia más ampliamente usada hoy para el aseguramiento estadístico de la calidad en la industria. La estrategia Seis Sigma fue popularizada originalmente por Motorola en la década de 1980 y "es una metodología rigurosa y disciplinada que usa datos y análisis estadísticos para medir y mejorar el desempeño operativo de una compañía, identificando y eliminando defectos en procesos de manufactura y servicios" [ISI08]. El término Seis Sigma se deriva

de seis desviaciones estándar —3.4 casos (defectos) por millón de ocurrencias—, lo que implica un estándar de calidad extremadamente alto. La metodología Seis Sigma define tres etapas fundamentales:

¿Cuáles son las etapas fundamentales de la metodología Seis Sigma?

- *Definir* los requerimientos del cliente y los que se le entregan, así como las metas del proyecto a través de métodos bien definidos de comunicación con el cliente.
- *Medir* el proceso existente y su resultado para determinar el desempeño actual de la calidad (recabar métricas para los defectos).
- *Analizar* las métricas de los defectos y determinar las pocas causas vitales.

Si se trata de un proceso de software existente que se requiere mejorar, Seis Sigma sugiere dos etapas adicionales:

- *Mejorar* el proceso, eliminando las causas originales de los defectos.
- *Controlar* el proceso para asegurar que el trabajo futuro no vuelva a introducir las causas de los defectos.

Estas etapas fundamentales y adicionales en ocasiones son conocidas como método DMAMC (definir, medir, analizar, mejorar y controlar).

Si una organización va a desarrollar un proceso de software (en vez de mejorar uno existente), a las etapas fundamentales se agregan las siguientes:

- *Diseñar* el proceso para 1) evitar las causas originales de los defectos y 2) cumplir los requerimientos del cliente.
- *Verificar* que el modelo del proceso en realidad evite los defectos y cumpla los requerimientos del cliente.

Esta variación en ocasiones es denominada método DMADV (definir, medir, analizar, diseñar y verificar).

El estudio detallado de Seis Sigma se deja a fuentes dedicadas a ese tema. Si el lector tiene interés al respecto, consulte [ISI08], [Pyz303] y [Sne03].

16.6 Confiabilidad del software

Cita:

"El precio inevitable de la confiabilidad es la simplicidad."

C. A. R. Hoare

No hay duda de que la confiabilidad de un programa de cómputo es un elemento importante de su calidad general. Si un programa falla repetida y frecuentemente en su desempeño, importa poco si otros factores de la calidad del software son aceptables.

La confiabilidad del software, a diferencia de muchos otros factores de la calidad, se mide y estima directamente mediante el uso de datos históricos del desarrollo. La *confiabilidad del software* se define en términos estadísticos como "la probabilidad que tiene un programa de cómputo de operar sin fallas en un ambiente específico por un tiempo específico" [Mus87]. Para ilustrar lo anterior, digamos que se estima que el programa *X* tiene una confiabilidad de 0.999 durante ocho horas de procesamiento continuo. En otras palabras, si el programa *X* fuera a ejecutarse 1 000 veces y requiriera un total de ocho horas de tiempo de procesamiento continuo (tiempo de procesamiento), es probable que operara correctamente (sin fallas) 999 veces.

Siempre que se trate de la confiabilidad del software, surge una pregunta crucial: ¿qué significa el término *falla*? En el contexto de cualquier análisis de la calidad y confiabilidad del software, la falla significa la falta de conformidad con los requerimientos del software. Pero, incluso con esta definición, hay gradaciones. Las fallas pueden ser leves o catastróficas. Una falla podría corregirse en segundos, mientras que otra tal vez requiera de varias semanas o meses de trabajo para ser corregida. Para complicar más el asunto, la corrección de una falla quizá dé como resultado la introducción de otros errores que a su vez originen otras fallas.

16.6.1 Mediciones de la confiabilidad y disponibilidad

PUNTO CLAVE

Los problemas de confiabilidad del software casi siempre pueden seguirse hasta encontrar defectos en el diseño o en la implementación.

Los primeros trabajos sobre confiabilidad del software trataban de extrapolar la teoría matemática de la confiabilidad del hardware a la predicción de la confiabilidad del software. La mayor parte de modelos relacionados con el hardware se abocan a la falla debida al uso, en lugar de a la que tiene su origen en los defectos de diseño. En el hardware, las fallas debidas al uso físico (por ejemplo, los efectos de temperatura, corrosión y golpes) son más probables que las debidas al diseño. Desafortunadamente, con el software ocurre lo contrario. En realidad, todas las fallas del software pueden rastrearse en problemas de diseño o de implementación; el uso (véase el capítulo 1) no entra en el escenario.

Ha habido un debate permanente acerca de la relación que existe entre los conceptos clave en la confiabilidad del hardware y su aplicabilidad al software. Aunque es posible establecer un vínculo irrefutable, es útil considerar algunos conceptos sencillos que se aplican a ambos elementos del sistema.

PUNTO CLAVE

Es importante observar que el tiempo medio entre fallas y otras medidas relacionadas se basa en tiempo del CPU, no en tiempo de reloj.

Si se considera un sistema basado en computadora, una medida sencilla de su confiabilidad es el *tiempo medio entre fallas* (TMEF):

$$\text{TMEF} = \text{TMPF} + \text{TMPR}$$

donde las siglas TMPF y TMPR significan *tiempo medio para la falla* y *tiempo medio para la reparación*,[2] respectivamente.

Muchos investigadores afirman que el TMEF es una medición más útil que otras relacionadas con la calidad del software que se estudian en el capítulo 23. En pocas palabras, a un usuario final le preocupan las fallas, no la cuenta total de defectos. Como cada defecto contenido en un programa no tiene la misma tasa de fallas, la cuenta total de defectos indica muy poco acerca de la confiabilidad del sistema. Por ejemplo, considere un programa que haya estado en operación durante 3 000 horas de procesador sin falla. Muchos defectos de este programa estarían sin detectar durante decenas de miles de horas antes de ser descubiertos. El TMEF de tales errores oscuros podría ser de 30 000 o hasta 60 000 horas de procesador. Otros defectos, no descubiertos, podrían tener una tasa de fallas de 4 000 a 5 000 horas. Aun si cada uno de los errores en esta categoría (los que tienen un TMEF largo) se eliminara, el efecto que tendrían sobre el software sería despreciable.

CONSEJO

Algunos aspectos de la disponibilidad (que no se estudian aquí) no tienen que ver con las fallas. Por ejemplo, la programación del tiempo fuera de operación (para funciones de apoyo) hace que el software no esté disponible.

Sin embargo, el TMEF puede ser problemático por dos razones: 1) proyecta un tiempo entre fallas, pero no da una tasa de fallas proyectada y 2) puede interpretarse mal, como la vida promedio, cuando *no* es esto lo que implica.

Una medición alternativa de confiabilidad es la de las *fallas en el tiempo* (FET): medición estadística de cuántas fallas tendrá un componente en mil millones de horas de operación. Por tanto, 1 FET es equivalente a una falla en cada mil millones de horas de operación.

Además de una medida de la confiabilidad, también debe desarrollarse otra para la disponibilidad. La *disponibilidad del software* es la probabilidad de que un programa opere de acuerdo con los requerimientos en un momento determinado de tiempo, y se define así:

$$\text{Disponibilidad} = \frac{\text{TMPF}}{\text{TMPF} + \text{TMPR}} \times 100\%$$

La medición del TMEF para la confiabilidad es igualmente sensible al TMPF y al TMPR. La medición de la disponibilidad es un poco más sensible al TMPR, que es una medición indirecta de la facilidad que tiene el software para recibir mantenimiento.

---

2 Aunque tal vez se requiera depurar (y hacer otras correcciones relacionadas) como consecuencia de la falla, en muchos casos el software funcionará de manera apropiada después de reiniciar, sin ningún otro cambio.

Cita:

"La seguridad de las personas debe ser la ley máxima."

Cicerón

16.6.2 Seguridad del software

La *seguridad del software* es una actividad del aseguramiento del software que se centra en la identificación y evaluación de los peligros potenciales que podrían afectarlo negativamente y que podrían ocasionar que falle todo el sistema. Si los peligros se identifican al principio del proceso del software, las características de su diseño se especifican de modo que los eliminen o controlen.

Como parte de la seguridad del software, se lleva a cabo un proceso de modelado y análisis. Inicialmente se identifican los peligros y se clasifican según su riesgo. Por ejemplo, algunos de los peligros asociados con un control de crucero basado en computadora para un automóvil podrían ser los siguientes: 1) ocasionar una aceleración incontrolada que no pudiera detenerse, 2) no responder a la presión en el pedal de frenado (porque se apague), 3) no encender cuando se active el interruptor y 4) perder o ganar velocidad poco a poco. Una vez identificados estos peligros en el nivel del sistema, se utilizan técnicas de análisis para asignar severidad y probabilidad de ocurrencia a cada uno.[3] Para ser eficaz, el software debe analizarse en el contexto de todo el sistema. Por ejemplo, un error sutil en la entrada de un usuario (las personas son componentes del sistema) podría ampliarse por una falla del software y producir datos de control que situaran equivocadamente un dispositivo mecánico. Si y sólo si se encontrara un único conjunto de condiciones ambientales externas, la posición falsa del dispositivo mecánico ocasionaría una falla desastrosa. Podrían usarse técnicas de análisis [Eri05], tales como árbol de fallas, lógica en tiempo real y modelos de red de Petri, para predecir la cadena de eventos que ocasionarían los peligros, así como la probabilidad de ocurrir que tendría cada uno de los eventos para generar la cadena.

Cita:

"No puedo imaginar ninguna condición que hiciera que esta nave se hundiera. La construcción naval moderna ha llegado más allá de eso".

E. I. Smith, capitán del *Titanic*

Una vez identificados y analizados los peligros, pueden especificarse requerimientos relacionados con la seguridad para el software. Es decir, la especificación contendría una lista de eventos indeseables y las respuestas deseadas del sistema ante ellos. Después se indicaría el papel del software en la administración indeseable de los mismos.

Aunque la confiabilidad y la seguridad del software están muy relacionadas, es importante entender la sutil diferencia entre ellas. La primera utiliza técnicas de análisis estadístico para determinar la probabilidad de que ocurra una falla del software. Sin embargo, la ocurrencia de una falla no necesariamente da como resultado un peligro o riesgo. La seguridad del software examina las formas en las que las fallas generan condiciones que llevan a un peligro. Es decir, las fallas no se consideran en el vacío, sino que se evalúan en el contexto de la totalidad del sistema basado en computadora y de su ambiente.

WebRef

En la dirección **www.safeware-eng.com/**, se encuentran varios artículos sobre seguridad del software.

El estudio exhaustivo de la seguridad del software está más allá del alcance de este libro. Si el lector está interesado en la seguridad del software y en otros aspectos relacionados, consulte [Smi05], [Dun02] y [Lev95].

16.7 Las normas de calidad ISO 9000[4]

Un *sistema de aseguramiento de la calidad* se define como la estructura organizacional, responsabilidades, procedimientos, procesos y recursos necesarios para implementar la administración de la calidad [ANS87]. Los sistemas de aseguramiento de la calidad se crean para ayudar a las organizaciones a asegurar que sus productos y servicios satisfagan las expectativas del con-

3 Este enfoque es similar a los métodos de análisis del riesgo descritos en el capítulo 28. La diferencia principal es el énfasis que se pone en aspectos de la tecnología en lugar de en los relacionados con el proyecto.

4 Esta sección, escrita por Michael Stovski, ha sido adaptada a partir de "Fundamentos de ISO 9000", libro de trabajo desarrollado para *Essential Software Engineering*, video desarrollado por R. S. Pressman & Associates, Inc. Se reimprime con su autorización.

sumidor gracias a que cumplan con sus especificaciones. Estos sistemas cubren una amplia variedad de actividades, que contemplan todo el ciclo de vida del producto, incluidos planeación, control, medición, pruebas e informes, así como la mejora de los niveles de calidad en todo el proceso de desarrollo y manufactura. La norma ISO 9000 describe en términos generales los elementos de aseguramiento de la calidad que se aplican a cualquier negocio, sin importar los productos o servicios ofrecidos.

Para registrarse en alguno de los modelos del sistema de aseguramiento de la calidad contenidos en la ISO 9000, por medio de auditores externos se revisan en detalle el sistema y las operaciones de calidad de una compañía, respecto del cumplimiento del estándar y de la operación eficaz. Después de un registro exitoso, el grupo de registro representado por los auditores emite un certificado para la compañía. Auditorías semestrales de supervisión aseguran el cumplimiento continuo de la norma.

WebRef

En la dirección **www.tantara.ab.ca/info.htm**, se encuentran muchos vínculos hacia los recursos de la norma ISO 9000/9001.

Los requerimientos esbozados por la norma ISO 9001:2000 se dirigen a temas tales como responsabilidad de la administración, sistema de calidad, revisión del contrato, control del diseño, documentación y control de datos, identificación del producto y su seguimiento, control del proceso, inspección y pruebas, acciones correctivas y preventivas, registros del control de calidad, auditorías internas de calidad, capacitación, servicio y técnicas estadísticas. A fin de que una organización de software se registre en la ISO 9001:2000, debe establecer políticas y procedimientos que cumplan cada uno de los requerimientos mencionados (y otros más), y después demostrar que sigue dichas políticas y procedimientos. Si el lector desea más información sobre la norma ISO 9001:2000, consulte [Ant06], [Mut03] o [Dob04].

INFORMACIÓN



La norma ISO 9001:2000

La descripción siguiente define los elementos básicos de la norma ISO 9001:2000. Información completa sobre la misma se obtiene en la Organización Internacional de Normas (**www.iso.ch**) y en otras fuentes de internet (como en **www.praxiom.com**).

Establecer los elementos de un sistema de administración de la calidad.
- Desarrollar, implementar y mejorar el sistema.
- Definir una política que ponga el énfasis en la importancia del sistema.

Documentar el sistema de calidad.
- Describir el proceso.
- Producir un manual de operación.
- Desarrollar métodos para controlar (actualizar) documentos.
- Establecer métodos de registro.

Apoyar el control y aseguramiento de la calidad.
- Promover la importancia de la calidad entre todos los participantes.
- Centrarse en la satisfacción del cliente.
- Definir un plan de calidad que se aboque a los objetivos, responsabilidades y autoridad.
- Definir mecanismos de comunicación entre los participantes.

Establecer mecanismos de revisión para el sistema de administración de la calidad.
- Identificar métodos de revisión y mecanismos de retroalimentación.
- Definir procedimientos para dar seguimiento.

Identificar recursos para la calidad, incluidos personal, capacitación y elementos de la infraestructura.
- Establecer mecanismos de control.
- Para la planeación
- Para los requerimientos del cliente
- Para las actividades técnicas (tales como análisis, diseño y pruebas)
- Para la vigilancia y administración del proyecto

Definir métodos de corrección.
- Evaluar datos y métricas de la calidad.
- Definir el enfoque para la mejora continua del proceso y la calidad.

16.8 EL PLAN DE ACS

El *Plan de ACS* proporciona un mapa de ruta para instituir el aseguramiento de la calidad del software. Desarrollado por el grupo de ACS (o por el equipo del software si no existe un grupo de ACS), el plan funciona como plantilla para las actividades de ACS que se instituyen para cada proyecto de software.

La IEEE [IEEE93] ha publicado una norma para el ACS. Ésta recomienda una estructura que identifica lo siguiente: 1) propósito y alcance del plan, 2) descripción de todos los productos del trabajo de ingeniería de software (tales como modelos, documentos, código fuente, etc.) que se ubiquen dentro del ámbito del ACS, 3) todas las normas y prácticas aplicables que se utilicen durante el proceso del software, 4) acciones y tareas del ACS (incluidas revisiones y auditorías) y su ubicación en el proceso del software, 5) herramientas y métodos que den apoyo a las acciones y tareas de ACS, 6) procedimientos para la administración de la configuración del software (véase el capítulo 22), 7) métodos para unificar las salvaguardas y para mantener todos los registros relacionados con el ACS y 8) roles y responsabilidades relacionados con la calidad del producto.

HERRAMIENTAS DE SOFTWARE

Administración de la calidad del software



Objetivos: El objetivo de las herramientas del ACS es ayudar al equipo del proyecto a evaluar y mejorar la calidad del producto del trabajo de software.

Mecánica: La mecánica de las herramientas varía. En general, el objetivo consiste en evaluar la calidad de un producto específico. Nota: Es frecuente que dentro de la categoría de herramientas para el ACS, se incluya una amplia variedad de herramientas para someter a prueba al software (véanse los capítulos 17 a 20).

Herramientas representativas[5]

ARM, desarrollada por la NASA (**state.gsfc.nasa.gov/tools/index.html**), proporciona mediciones que se utilizan para evaluar la calidad de un documento de requerimientos de software.

QPR ProcessGuide and Scorecard, desarrollada por QPR Software (**www.qpronline.com**), da apoyo para establecer Seis Sigma y otros enfoques de administración de la calidad.

Quality Tools and Templates, desarrollada por iSixSigma (**www.isixsigma.com/tt/**), describe un amplio abanico de herramientas y métodos útiles para la administración de la calidad.

NASA Quality Resources, desarrollada por el Centro Coddard de Vuelos Espaciales (**sw-assurance.gsfc.nasa.gov/index.php**), contiene formatos, plantillas, listas de verificación y herramientas que son útiles para el ACS.

16.9 RESUMEN

El aseguramiento de la calidad del software es una actividad sombrilla de la ingeniería de software que se aplica en cada etapa del proceso del software. El ACS incluye procedimientos para la aplicación eficaz de métodos y herramientas, supervisa las actividades de control de calidad, tales como las revisiones técnicas y las pruebas del software, procedimientos para la administración del cambio, y procedimientos para asegurar el cumplimiento de las normas y mecanismos de medición y elaboración de reportes.

Para llevar a cabo el aseguramiento de la calidad del software de manera adecuada, deben recabarse, evaluarse y divulgarse datos sobre el proceso de la ingeniería de software. Los métodos estadísticos aplicados al ACS ayudan a mejorar la calidad del producto y del proceso de software mismo. Los modelos de confiabilidad del software amplían las mediciones, lo que permite que los datos obtenidos acerca de los defectos se extrapolen hacia tasas de falla proyectadas y hacia la elaboración de pronósticos de confiabilidad.

En resumen, deben tomarse en cuenta las palabras de Dunn y Ullman [Dun82]: "El aseguramiento de la calidad del software es el mapeo de los preceptos administrativos y de las disciplinas de diseño del aseguramiento de la calidad, en el ámbito administrativo y tecnológico aplicable a la ingeniería de software." La capacidad de asegurar la calidad es la medida de una

5 Las herramientas mencionadas aquí no son obligatorias, sino una muestra de las que hay en esta categoría. En la mayoría de casos, los nombres de las herramientas son marcas registradas por sus respectivos desarrolladores.

disciplina madura de la ingeniería. Cuando el mapeo se lleva a cabo con éxito, el resultado es una ingeniería de software madura.

Problemas y puntos por evaluar

16.1. Algunas personas afirman que "el control de la variación es el corazón del control de calidad". Como todo programa que se crea es diferente de cualquier otro programa, ¿cuáles son las variaciones que se buscan y cómo se controlan?

16.2. ¿Es posible evaluar la calidad del software si el cliente cambia continuamente lo que se supone que debe hacerse?

16.3. La calidad y confiabilidad son conceptos relacionados, pero difieren en lo fundamental por varias razones. Analice las diferencias.

16.4. ¿Un programa puede corregirse y aún así ser confiable? Explique su respuesta.

16.5. ¿Un programa puede corregirse y tener buena calidad? Explique lo que responda.

16.6. ¿Por qué es frecuente que haya tensiones entre el grupo de ingeniería de software y el del aseguramiento de la calidad? ¿Es saludable eso?

16.7. El lector tiene la responsabilidad de mejorar la calidad del software en su organización. ¿Qué es lo primero que debe hacer? ¿Qué es lo siguiente?

16.8. Además de contar los errores y defectos, ¿hay otras características cuantificables de software que impliquen calidad? ¿Cuáles son y cómo podrían medirse directamente?

16.9. El concepto del tiempo medio para la falla del software es objeto de críticas. Explique por qué.

16.10. Considere dos sistemas cuya seguridad sea crítica y que estén controlados por computadora. Enliste al menos tres peligros que se relacionen directamente con fallas del software.

16.11. Obtenga una copia de las normas ISO 9001:2000 e ISO 9000-3. Prepare una presentación que analice tres requerimientos de ISO 9001 y la forma en la que se apliquen en el contexto del software.

Lecturas y fuentes de información adicionales

Los libros de Hoyle (*Quality Management Fundamentals*, Butterworth-Heinemann, 2007), Tian (*Software Quality Engineering*, Wiley-IEEE Computer Society Press, 2005), El Emam (*The ROI from Software Quality*, Auerbach, 2005) y Horch (*Practical Guide to Software Quality Management*, Artech House, 2003), y Nance y Arthur (*Managing Software Quality*, Springer, 2002) son presentaciones excelentes en el nivel de administración acerca de los beneficios de los programas formales de aseguramiento de la calidad del software de computadora. Las obras de Deming [Dem86], Juran (*Juran on Quality by Design*, Free Press, 1992) y Crosby ([Cro79], así como *Quality is Still Free*, McGraw-Hill, 1995) no se abocan al software, pero son una lectura obligada para los altos directivos que tengan responsabilidades en el desarrollo del software. Gluckman y Roome (*Everyday Heroes of the Quality Movement*, Dorset House, 1993) humanizan los aspectos de la calidad a través de la historia de los actores participantes en el proceso. Kan (*Metrics and Models in Software Quality Engineering*, Addison-Wesley, 1995) presenta un enfoque cuantitativo de la calidad del software.

Los libros de Evans (*Total Quality: Management, Organization and Strategy*, 4a. ed., South Western College Publishing, 2004), Bru (*Six Sigma for Managers*, McGraw-Hill, 2005) y Dobb (*ISO 9001:2000 Quality Registration Step-by-Step*, 3a. ed., Butterworth-Heinemann, 2004) son representativos de los muchos que se han escrito sobre Seis Sigma e ISO 9001:2000, respectivamente.

Pham (*System Software Reliability,* Springer, 2006), Musa (*Software Reliability Engineering: More Reliable Software, Faster Development and Testing*, 2a. ed., McGraw-Hill, 2004) y Peled (*Software Reliability Methods*, Springer, 2001) proporcionan guías prácticas que describen los métodos para medir y analizar la confiabilidad del software.

Vincoli (*Basic Guide to System Safety*, Wiley 2006), Dhillon (*Engineering Safety*, World Scientific Publishing Co., Inc., 2003), Hermann (*Software Safety and Reliability*, Wiley-IEEE Computer Society Press, 2000), Storey (*Safety-Critical Computer Systems*, Addison-Wesley, 1996) y Leveson [Lev95] aportan los análisis más exhaustivos que se hayan publicado hasta la fecha acerca de la seguridad del software y del sistema. Además, Van

der Meulen (*Definitions for Hardware and Software Safety Engineers*, Springer-Verlag, 2000) ofrece un compendio completo de conceptos y términos importantes para la confiabilidad y la seguridad; Gartner (*Testing Safety-Related Software*, Springer-Verlag, 1999) ofrece una guía especializada para probar sistemas cuya seguridad sea crítica; Friedman y Voas (*Software Assesment: Reliability Safety and Testability*, Wiley, 1995) proveen modelos útiles para evaluar la confiabilidad y la seguridad. Ericson (*Hazard Analysis Techniques for System Safety*, Wiley, 2005) estudia el dominio cada vez más importante del análisis de los peligros.

En internet, hay una amplia variedad de fuentes de información sobre el aseguramiento de la calidad del software y otros temas relacionados. En el sitio web del libro, **www.mhhe.com/engcs/compsci/pressman/professional/olc/ser.htm**, existe una lista actualizada de referencias existentes en la red mundial que son relevantes para el ACS.